Manipulating and Visualizing Data

Why do we need an extra week on this?

Base `R` is limited for data manipulation and visualization

It can be:

- Hard to read, especially across languages
- Inefficient (many lines of code to perform a common operation)

What is Base R

"Base R" = What is available in R with no added packages.

Note: There are benefits to Base R.

- "Lightweight" (no additional time/space to install)
- Especially stable across updates

What is the alternative?

"Tidyverse" describes a suite of `R` packages supported by Posit

Information at: https://tidyverse.org

Associated with "R for Data Science": https://r4ds.hadley.nz

Install it!

What will we learn about the "Tidyverse"?

We will learn about:

- `dplyr`: Helps filter and aggregate data
- `ggplot2`: Helps create nice plots

Getting Ready to Explore dplyr

We're going to use dplyr to summarize data on flights that departed New York City from JFK, LGA, or EWR in 2013.

```
library(nycflights13)
library(dplyr)
flights <- nycflights13::flights
```

The Flights Data is a tibble

```
str(flights)
```

```
tibble [336,776 x 19] (S3: tbl_df/tbl/data.frame)
 $ year          : int [1:336776] 2013 2013 2013 2013 2013
 $ month         : int [1:336776] 1 1 1 1 1 1 1 1 1 1 ...
 $ day           : int [1:336776] 1 1 1 1 1 1 1 1 1 1 ...
 $ dep_time      : int [1:336776] 517 533 542 544 554 554 5
 $ sched_dep_time: int [1:336776] 515 529 540 545 600 558 6
 $ dep_delay     : num [1:336776] 2 4 2 -1 -6 -4 -5 -
3 -3 -2 ...
 $ arr_time      : int [1:336776] 830 850 923 1004 812 740
 $ sched_arr_time: int [1:336776] 819 830 850 1022 837 728
 $ arr_delay     : num [1:336776] 11 20 33 -18 -
25 12 19 -14 -8 8 ...
 $ carrier       : chr [1:336776] "UA" "UA" "AA" "B6" ...
 $ flight        : int [1:336776] 1545 1714 1141 725 461 16
 $ tailnum       : chr [1:336776] "N14228" "N24211" "N619AA
 $ origin        : chr [1:336776] "EWR" "LGA" "JFK" "JFK"
```

Converting the Flights Data to a Data Frame

To avoid learning about a new data structure, we'll convert the `tibble` to a data frame.

```
flights <- as.data.frame(flights)
```

The `filter` Function

Allows us to easily subset data according to the values that a variable takes on

```
filter(flights, month == 1, day == 1)
```

Previously we did this by indexing using logical vectors

```
flights[flights$month == 1 &
          flights$day == 1, ]
```

Filtering on Multiple Values

Just like we saw before when learning about logical vectors, there's multiple ways to do this.

```
filter(flights, month == 11 | month == 12)
filter(flights, month %in% c(11, 12))
```

Missing Value Handling

Missing values (`NA`) are handled in natural ways by `filter`

```
filter(flights,
       !(arr_delay > 120 |
           dep_delay > 120))
```

Before, we would have needed to type

```
flights[!is.na(flights$arr_delay) &
          !is.na(flights$dep_delay) &
          flights$arr_delay <= 120 &
          flights$dep_delay <= 120, ]
```

Filtering Using Intervals

The `between` function with `left` and `right` arguments allow us to filter on variables being in an interval.

```
filter(flights, arr_delay <= 120,
       between(x = dep_delay, left = 80, right = 120))
```

This is much uglier with base `R`!

```
flights[!is.na(flights$arr_delay) &
          !is.na(flights$dep_delay) &
          flights$arr_delay <= 120 &
          flights$dep_delay >= 80 &
          flights$dep_delay <= 120, ]
```

Letting Missing Values Through

Sometimes, we want to retain some observations with missing values.

```
filter(flights, is.na(arr_delay) |
         arr_delay <= 120,
       between(x = dep_delay, left = 80, right = 120))
```

In base `R` this would have been

```
flights[!is.na(flights$dep_delay) &
          flights$arr_delay <= 120 &
          flights$dep_delay >= 80 &
          flights$dep_delay <= 120, ]
```

Activities to Try Yourself

Find all flights that were:

- Delayed two or more hours
- Flew to Houston
- Were operated by United, American, or Delta
- Departed in summer (July, August, and September)
- Arrived more than two hours late, but didn't leave late
- Delayed at least an hour, but made up over 30 minutes in flight
- Departed between midnight and 6:00AM (inclusive)

The arrange Function

The arrange function allows us to sort data frames easily by multiple variables

```
arrange(flights, year, month, day)
```

Again, we could have done this in base R

```
flights[order(flights$year, flights$month, flights$day), ]
```

Sorting in Reverse

```
arrange(flights, desc(dep_delay))
```

Note: This is starting to look like SQL!

The `select` Function

The `select` function allows us to subset variables

```
select(flights, year, month, day)
```

```
select(flights, year:day)
```

```
select(flights, -(year:day))
```

```
select(flights, time_hour, air_time, everything())
```

The rename Function

The rename function lets us subset multiple variables at once.

```
renamed <- rename(flights, tail_num = tailnum)
```

This would have been much more code in base R!

```
renamed <- flights
renamed$tail_num <- flights$tailnum
renamed$tailnum <- NULL
```

```
renamed <- flights
names(renamed)[names(renamed) == "tailnum"] <-
  "tail_num"
```

The mutate Function

The mutate function makes it easy to construct new variables.

```
flightsnew <- mutate(flights,
                     gain = dep_delay - arr_delay,
                     speed = distance / air_time * 60)
```

This would also be a pain in base R

```
flightsnew <- flights
flightsnew$gain <- flightsnew$dep_delay -
  flightsnew$arr_delay
flightsnew$speed <- flightsnew$distance/
  flightsnew$air_time * 60
```

The mutate Function

The mutate function even allows you to use newly defined variables to define other new variables

```
flightsnew2 <- mutate(flights,
                      gain = dep_delay - arr_delay,
                      hours = air_time / 60,
                      gain_per_hour = gain / hours)
```

This would **really** be a pain in base R

```
flightsnew2 <- flights
flightsnew2$gain <- flightsnew2$dep_delay - flightsnew2$ar
flightsnew2$hours <- flightsnew2$air_time/60
flightsnew2$gain_per_hour <-  flightsnew2$gain/flightsnew2
```

The transmute Function

The transmute function is similar to mutate, but drops variables that aren't modified

```
flightsnew2 <- transmute(flights,
                         gain = dep_delay - arr_delay,
                         hours = air_time / 60,
                         gain_per_hour = gain / hours)
head(flightsnew2)
```

```
  gain    hours gain_per_hour
1   -9 3.783333     -2.378855
2  -16 3.783333     -4.229075
3  -31 2.666667    -11.625000
4   17 3.050000      5.573770
5   19 1.933333      9.827586
6  -16 2.500000     -6.400000
```

Activities to Try Yourself

Using the `filter` function, identify flights with missing departure delay.

What's going on?

The summarize Function

The summarize function allows us to create aggregate summaries

```
summarise(flights, delay = mean(dep_delay, na.rm = TRUE))
```

```
     delay
1 12.63907
```

This was easy to do in base R

```
mean(flights$dep_delay, na.rm = TRUE)
```

```
[1] 12.63907
```

Aggregating within Groups

When combined with `group_by`, we can also construct aggregate summaries within levels of other variables

```
bymy <- summarise(group_by(flights, year, month),
                  delay = mean(dep_delay, na.rm = TRUE))
```

Doing this in base `R` requires the aggregate function

```
bymy <- aggregate(flights$dep_delay,
                  list("year" = flights$year,
                       "month" = flights$month),
                  mean, na.rm = TRUE)
```

Aggregating within Groups - Comparison

The output of summarize is much nicer

```
summarise(group_by(flights, year, month),
          delay = mean(dep_delay, na.rm = TRUE))[1, ]
```

```
# A tibble: 1 x 3
# Groups:   year [1]
   year month delay
  <int> <int> <dbl>
1  2013     1  10.0
```

Doing this in base R requires the aggregate function

```
aggregate(flights$dep_delay,
          list("year" = flights$year,
               "month" = flights$month),
          mean, na.rm = TRUE)[1, ]
```

```
  year month        x
1 2013     1 10.03667
```

Creating Multiple Aggregate Variables

This becomes **much** easier with `dplyr` than base `R`

```
bymy <- summarise(group_by(flights, year, month),
                  delay = mean(dep_delay, na.rm = TRUE),
                  dist = mean(distance, na.rm = TRUE),
                  count = n())
```

Creating Multiple Aggregate Variables (Base)

In base R, we need to create the new variables one at a time and then merge them together. Ugh!

```
dep_delaymeans <- aggregate(flights$dep_delay,
                            list("year" = flights$year,
                                 "month" = flights$month),
                            mean, na.rm = TRUE)
colnames(dep_delaymeans)[ncol(dep_delaymeans)] <- "dep_delay"
distmeans <- aggregate(flights$dist,
                                 list("year" = flights$year,
                                      "month" = flights$month),
                                 mean, na.rm = TRUE)
colnames(distmeans)[ncol(distmeans)] <- "dist"
distcounts <- aggregate(flights$dist,
                                 list("year" = flights$year,
                                      "month" = flights$month),
                                 length)
colnames(distcounts)[ncol(distcounts)] <- "n"
bymy <- merge(distmeans, dep_delaymeans, by = c("month", "year"))
bymy <- merge(bymy, distcounts, by = c("month", "year"))
```

A Diversion on Merging

We learned about merging (joining) in SQL. What about R?

We use the `merge` function.

```
bymy <- merge(bymy, distcounts, by = c("month", "year"),
              all.x = TRUE, all.y = TRUE)
```

The arguments `all.x` and `all.y` refer to the first and second data frames provided and determine if it is a left, right, or inner join.

Creating Aggregate Variables without Aggregating Data

Combining `mutate` with `group_by` lets us add summaries to the original data

```
mutate(group_by(flights, year, month),
       dep_delay_bymonth = mean(dep_delay, na.rm = TRUE))
```

In base `R`, this always involves multiple steps

```
delaypermonth <- aggregate(flights$dep_delay,
                           list("month" = flights$month),
                           mean, na.rm = TRUE)
names(delaypermonth)[ncol(delaypermonth)] <-
  "dep_delay_bymonth"
flightsnew5 <- merge(flights, delaypermonth, by = "month")
```

Grouping the Data

Sometimes, it can be beneficial to group the data as a separate command before applying "summarizeormutate"'

```
flights <- group_by(flights, dest)
delay <- summarise(flights,
                   count = n(),
                   dist = mean(distance, na.rm = TRUE),
                   delay = mean(arr_delay, na.rm = TRUE))
delay <- filter(delay, count > 20, dest != "HNL")
```

Note: Ungroup after with `flights <- ungroup(flights)`!! Otherwise any use of `summarize`, `mutate`, or `transmute` after will use the grouping you previously provided.

Aggregating at Different Levels

For a single analysis, we may want to aggregate at multiple levels

```
flights <- group_by(flights, dest)
delay <- summarise(flights,
                   count = n(),
                   dist = mean(distance, na.rm = TRUE),
                   delay = mean(arr_delay, na.rm = TRUE))
delay <- filter(delay, count > 20, dest != "HNL")
```

The Pipe Operator

A big selling point of using `dplyr` is the incorporation of the pipe operator `|>`

- Originated from an external package, was %>%
- `|>` is the base `R` equivalent of %>%, is now preferred over %>%

Use of |>

The pipe operator |> passes output from one function to another function as the first argument.

```
delay <- flights |>
  group_by(dest) |>
  summarize(count = n(),
                    dist = mean(distance, na.rm = TRUE),
                    delay = mean(arr_delay, na.rm = TRUE))
  filter(count > 20, dest != "HNL")
```

To Summarize

We have learned about:

- `filter`
- `arrange`
- `select`
- `rename`
- `mutate`
- `transmute`